

Paper-to-Question: An Automated Pipeline for Generating Decision-Relevant Forecasting Questions from Research Papers

Hauke Hillebrandt

July 6, 2026

Abstract

We present an automated pipeline that transforms research papers into decision-relevant binary forecasting questions with probabilistic estimates. The pipeline’s product is better *questions*: probability accuracy on any single question is a commodity led by specialist forecasters and frontier labs, and is not what an automated paper-to-question system competes on. The system implements 16 core stages—augmented with two optional cross-model second-opinion stages—combining large language model (LLM) agents with structured quality gates: text extraction, chunked proto-question generation, quality and priority filtering (an adapted Importance–Tractability–Neglectedness framework augmented with temporal urgency heuristics), cross-project deduplication, adversarial peer review, ensemble forecasting, question decomposition, probabilistic component analysis, multi-method reconciliation, per-question report pages, and optional cross-model second opinions (a local-CLI critique and a gated deep-research re-forecast). Built on the FutureSearch platform [Bosse et al., 2026], the pipeline processes multiple source papers in parallel and produces interactive HTML reports aggregating forecasts across projects.

Our headline result is that the multi-stage pipeline produces better questions than a single model pass: in a blinded comparison the pipeline’s question scored 80.0 versus a one-shot model’s 60.6 mean, and was ranked first by all three judges. A second controlled result concerns *benchmark over-indexing*—the tendency to generate “will model X hit Y% on benchmark Z” questions rather than decision-relevant real-world outcomes. A same-seed experiment traced this to a single pro-benchmark steer line in the original prompt rather than to intrinsic model bias: removing that line dropped the benchmark share from 42% to 17% (one-shot, 12 questions per arm), and a corpus measurement found 22% of generated proto-questions benchmark-flavored under the original (v1) prompts versus 0% under a later, heavily penalized rewrite (v2); the production prompts are now a de-steered neutral set that keeps the steer removal but drops the penalties.

We report initial deployments across eight source papers spanning five research domains (cybersecurity, AI R&D automation, biosecurity, AI governance, and AI-ownership economics), yielding 22 published forecasting questions from a stored pool of 178 proto-questions. A post-deployment audit of the stored artifacts shows that most early quality gates rarely bind in the narrow per-paper funnel: quality is enforced chiefly by adversarial *revision* (10 of 16 audited questions substantively rewritten, none killed) and, in the forecasting stages, by an inflate-then-correct dynamic (+8.7pp mean shift at decomposition, −12.4pp at reconciliation). As a secondary result—forecast augmentation is not the pipeline’s goal, only evidence that the plumbing works—a variance-controlled experiment shows that feeding a deep-research report into the second-opinion stage shifts a forecast by roughly +10pp beyond CLI sampling noise. We summarize these findings and outline a consolidated five-pass architecture as future work. Source code and the interactive reports are public.¹

¹Code: https://github.com/HaukeHillebrandt/forecasting_pipeline.
haukehillebrandt.github.io/forecasting-reports.

Reports: [https://](https://haukehillebrandt.github.io/forecasting-reports)

1 Introduction

Forecasting is a critical tool for evidence-based policy and decision-making [Tetlock and Gardner, 2015]. Yet the process of generating well-formed forecasting questions from domain research remains largely manual: analysts must read papers, identify falsifiable claims, formulate precise binary questions with unambiguous resolution criteria, verify feasibility, and finally produce probability estimates. This is slow, expensive, and difficult to scale across multiple research domains.

Recent advances in LLM-based agentic systems suggest that much of this process can be automated [Halawi et al., 2024, Schoenegger et al., 2024]. Bosse et al. [2026] demonstrated that LLM agents equipped with web search and structured prompting can generate forecasting questions and produce calibrated probability estimates that approach the accuracy of experienced human forecasters. Their FutureSearch system provides primitives—single-agent generation, parallel agent mapping, ranking, forecasting, and deduplication—that can be composed into complex workflows.

We build on this work to construct a fully automated *paper-to-question pipeline*: given a research paper (or any source text), the system extracts falsifiable claims, generates candidate forecasting questions, applies multi-stage quality filtering and adversarial review, produces probability estimates using ensemble methods, and decomposes high-priority questions into component sub-forecasts that are then reconciled into final estimates. Our thesis is that the valuable output is the *questions*, not the probabilities: turning a dense research paper into a small set of well-formed, decision-relevant, resolvable forecasting questions is the hard editorial work that scales poorly by hand, whereas probability accuracy on any given question is a commodity already led by specialist forecasters and frontier labs. Accordingly, our primary evidence (Section 7.4) is a blind comparison of *question quality*; the forecasting machinery is reported and evaluated, but the forecast-augmentation results (Section 7.6) are secondary. The pipeline is designed to be:

1. **End-to-end**: from PDF input to interactive HTML report with no manual intervention.
2. **Quality-controlled**: multiple review stages screen for poorly-formed, ambiguous, or trivial questions. In practice, the audit in Section 8.1 shows quality is enforced less by hard gates—most of which rarely bind—than by refinement and adversarial revision.
3. **Scalable**: adaptive effort allocation directs computational resources toward high-priority questions.
4. **Multi-project**: a global deduplication and aggregation layer prevents redundancy across independent research domains.

We deploy the pipeline across eight source papers spanning cybersecurity, AI R&D automation, biosecurity, AI governance, and AI-ownership economics, generating 22 published forecasting questions from a stored pool of 178 proto-questions (148 under the original v1 prompts and 30 under the later v2 prompts; see Section 7). Because the prompt set changed between deployments, we scope quantitative claims to the relevant prompt era throughout.

2 Related Work

Forecasting tournaments and platforms. Structured forecasting competitions [Tetlock and Gardner, 2015] and prediction markets [Arrow et al., 2008] have demonstrated that aggregated probabilistic judgments can outperform individual expert opinion. Platforms such as Metaculus, Polymarket, and Good Judgment Open provide infrastructure for question creation and crowd-sourced forecasting, but question generation remains a manual editorial process.

LLM-based forecasting. Halawi et al. [2024] showed that LLMs with retrieval-augmented generation can approach human forecaster accuracy on binary questions. Schoenegger et al.

[2024] found that ensembles of LLM forecasters can match or exceed crowd forecasts. Bosse et al. [2026] introduced FutureSearch, a platform combining LLM agents with web search for automated question generation and forecasting, providing the SDK primitives on which our pipeline is built.

Automated question generation. Prior work on automated question generation has focused primarily on educational settings [Kurdi et al., 2020] or reading comprehension benchmarks. Our work differs in targeting *forecasting questions*—binary questions about future events with explicit resolution criteria, deadlines, and probability estimates.

Priority frameworks. The Importance–Tractability–Neglectedness (ITN) framework [Open Philanthropy, 2016] is widely used in effective altruism and policy analysis to prioritize research questions. We extend this framework with temporal urgency heuristics from Ord [2020]—Soon, Sudden, and Sharp—to produce a six-dimensional priority score that guides resource allocation within the pipeline.

3 System Overview

The pipeline consists of 16 core stages organized into four phases: *generation* (Stages 01–02d), *refinement and verification* (Stages 03–05), *forecasting* (Stages 06–06d), and *reporting* (Stages 06e, 07q, 07–08). Two optional cross-model second-opinion stages (06f and 06g, described in Section 4.13) can be inserted before reporting. Figure 1 shows the complete data flow; Table 3 lists every stage.

3.1 Design Principles

Progressive filtering. The pipeline generates many candidate questions cheaply, then applies increasingly expensive quality gates. This funnel architecture—inspired by information retrieval cascades [Wang et al., 2011]—ensures that expensive operations (adversarial review, ensemble forecasting, deep decomposition) are only applied to high-quality candidates.

Adaptive effort allocation. Computational effort scales with question priority. A composite ITNSSS score (defined in Section 4.4) determines whether each question receives HIGH, MEDIUM, or LOW effort at each stage, reducing cost without sacrificing quality on important questions.

Multi-method forecasting. Rather than relying on a single probability estimate, the pipeline produces up to four independent estimates per question: (1) an ensemble forecast, (2) a decomposition-informed re-forecast, (3) a mechanically combined component forecast, and (4) a holistic reconciled estimate. Divergence between methods signals uncertainty.

4 Methodology

4.1 Stage 01: Text Extraction

Source documents (PDF or plain text) are processed using PyPDF2 with a sanitization pipeline that removes binary artifacts (embedded images, base64 data), drops lines with >20% non-printable characters, and preserves page boundaries. The output is a clean plain-text representation of the source paper.

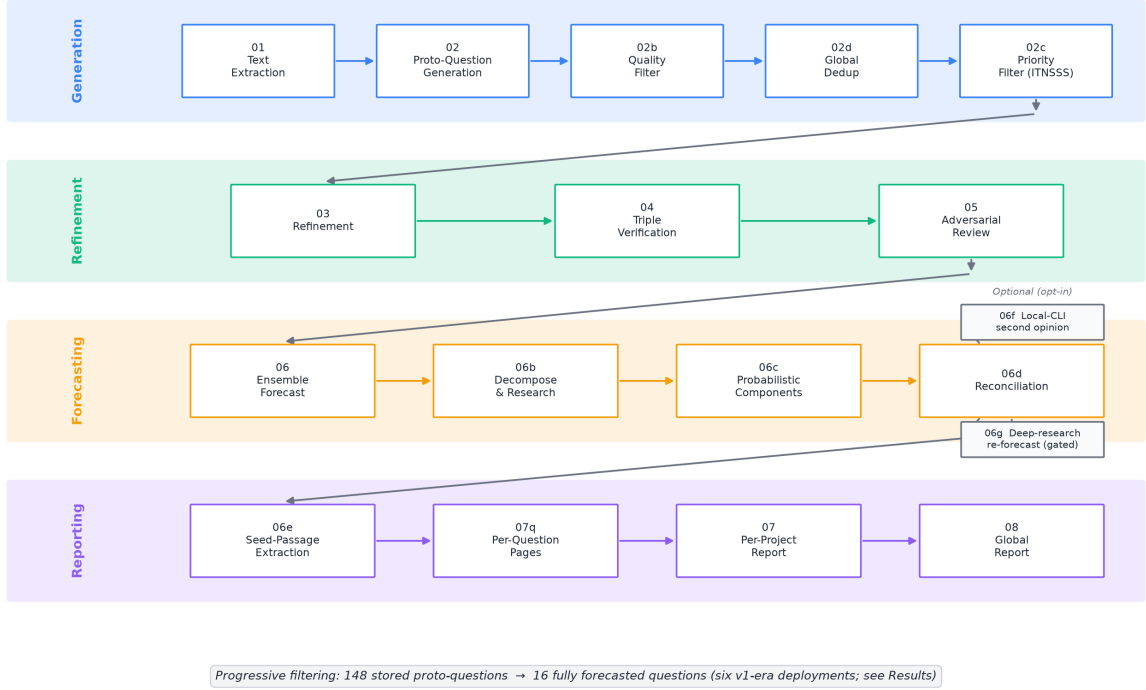


Figure 1: Pipeline architecture. The 16 core stages are organized into four phases: generation and filtering (blue), refinement and verification (green), forecasting and decomposition (orange), and reporting (purple); two optional cross-model second-opinion stages (06f, 06g) branch off the forecasting phase. Questions are progressively filtered from 148 stored proto-questions down to 16 fully forecasted questions across the six v1-era deployments (Section 7).

4.2 Stage 02: Proto-Question Generation

Long papers are split into chunks of approximately 4,000 tokens ($\approx 16,000$ characters) using a section-aware chunking algorithm. The chunker identifies section boundaries via regex patterns matching markdown headers, numbered sections, and named sections (Abstract, Introduction, Methods, Results, Discussion, References). Large sections are recursively split at paragraph boundaries; small adjacent sections are merged. This ensures coverage of the full paper while respecting semantic boundaries.

For each chunk, a FutureSearch `single_agent` with web search access generates $k = 10$ candidate proto-questions (configurable via `QUESTIONS_PER_CHUNK`). The agent is instructed to extract falsifiable claims and formulate binary questions with explicit resolution deadlines. Each question includes a rationale grounding it in the source text and a paper reference to the specific section or claim. The generation prompt has passed through three revisions (detailed in Section 6): the original *v1* prompt, which carried a pro-benchmark steer; a heavily penalized *v2* rewrite; and the current de-steered neutral prompt now in production. The deployments reported in Section 7 were produced under *v1* (six papers) and *v2* (two papers).

An inline quality scoring step (using `rank()`) selects the top $m = 2$ questions per chunk (`TOP_PER_CHUNK`), yielding approximately $2 \times |\text{chunks}|$ proto-questions per paper.

4.3 Stage 02b: Quality Filtering

All proto-questions from Stage 02 are scored on a 0–100 scale using FutureSearch’s `rank()` operation with three criteria:

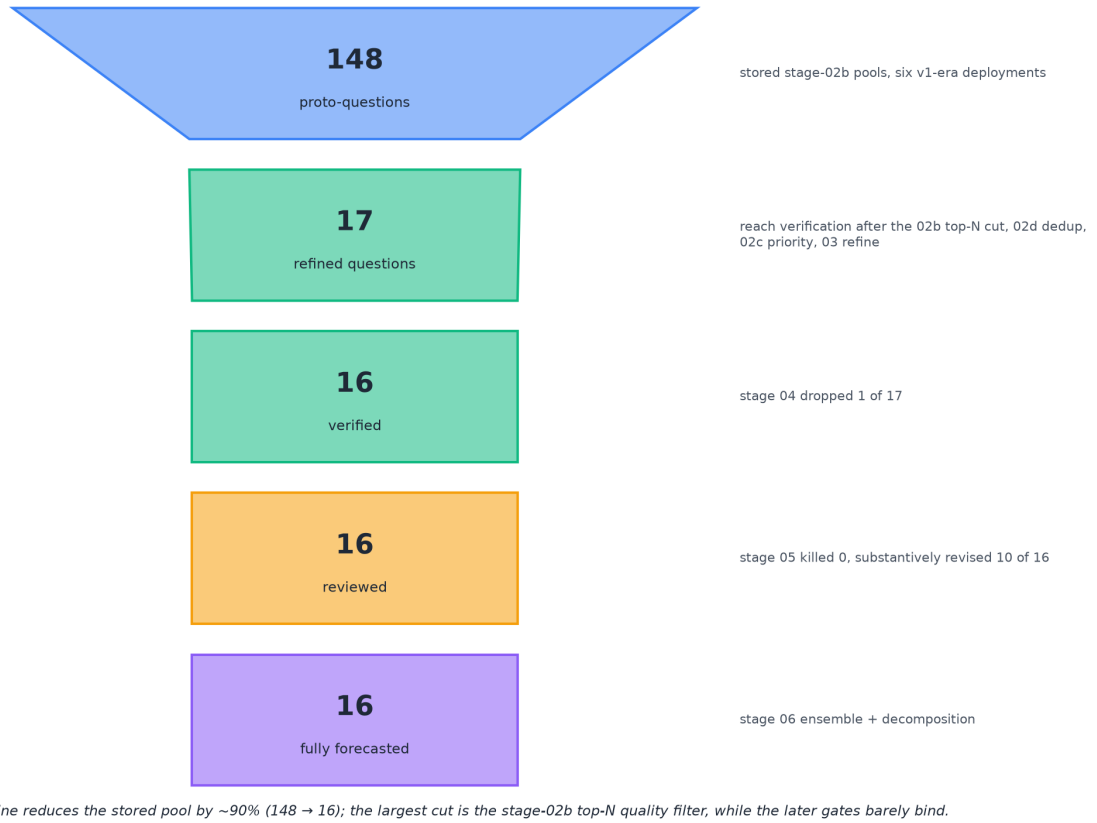


Figure 2: Filtering funnel (audited counts, six v1-era deployments). The stored pool of 148 proto-questions is reduced to 16 fully forecasted questions. The dominant cut is the stage-02b top- N quality filter; the later gates barely bind—global deduplication (02d) removed 0 duplicates, verification (04) dropped 1 of 17, and adversarial review (05) killed 0 while substantively revising 10 of 16. All counts are re-derived from the stored artifacts (Section 8.1).

1. **High entropy:** Is the question’s outcome genuinely uncertain?
2. **Data availability:** Can resolution be determined from publicly observable sources?
3. **Difficulty:** Does the question require genuine forecasting effort (not trivially answerable)?

The top N questions (default $N = 5$, configurable via `QUALITY_FILTER_TOP_N`) advance. An interactive HTML review report is generated for human oversight.

4.4 Stage 02c: Priority Filtering (ITNSSS)

Surviving questions are scored on six dimensions using FutureSearch `agent_map()` with web search. Each dimension produces a 0–100 score:

- **Importance** ($w = 0.25$): Decision-relevance within the project scope.
- **Neglectedness** ($w = 0.25$): Marginal information value—how much existing research covers this question.
- **Tractability** ($w = 0.20$): Feasibility of producing a meaningful forecast given available data.
- **Soon** ($w = 0.10$): Temporal urgency—how quickly the question will resolve.
- **Sudden** ($w = 0.10$): Whether the outcome is a discrete event vs. gradual change.
- **Sharp** ($w = 0.10$): Absence of warning signals before resolution.

The composite priority score is:

$$\text{priority} = 0.25 \cdot I + 0.25 \cdot N + 0.20 \cdot T + 0.10 \cdot S_{\text{soon}} + 0.10 \cdot S_{\text{sudden}} + 0.10 \cdot S_{\text{sharp}} \quad (1)$$

The first three dimensions (Importance, Tractability, Neglectedness) follow the ITN framework commonly used in cause prioritization [Open Philanthropy, 2016]. The latter three operationalize the “Soon, Sudden, Sharp” heuristics from Ord [2020], which characterize risks that warrant urgent attention because they may materialize quickly, discretely, and without precursors.

A *topic diversity constraint* prevents cluster dominance: the agent assigns each question a topic cluster label, and at most $c = 3$ questions per cluster advance (configurable via `MAX_PER_CLUSTER`).

4.5 Stage 02d: Global Deduplication

Before investing in refinement, proto-questions are checked against a global repository of all questions from prior pipeline runs across all projects. Two modes are available:

- **Local** (default, free): Case-insensitive string similarity using Python’s `difflib.SequenceMatcher` with threshold $\tau = 0.70$. The threshold is set higher than typical text similarity tasks because proto-questions share structural patterns (“Will X by December 31, 2027?”) that inflate similarity scores.
- **Semantic** (optional, ~\$0.50): FutureSearch’s `dedupe()` operation with a custom equivalence relation that identifies questions forecasting the same event even when differently worded.

4.6 Stage 03: Question Refinement

Each surviving proto-question is refined by a FutureSearch agent with web search into a full forecasting question comprising: (1) the question text, (2) contextual background, (3) explicit unambiguous resolution criteria, and (4) resolution sources (specific URLs or data sources where the outcome can be determined).

4.7 Stage 04: Triple Verification

Refined questions undergo three parallel verification checks:

Ambiguity scoring. A `rank()` operation scores each question on a 0–100 scale assessing whether 10 independent readers would agree on the outcome given the resolution criteria. Questions scoring below 60 are rejected.

Resolvability verification. An `agent_map()` operation checks a four-point checklist: (1) concrete observable event, (2) explicit resolution date, (3) publicly available evidence, (4) exhaustive YES/NO outcome space. Questions must receive “probably yes” or “very certainly yes” to advance.

Triviality screening. A separate agent produces a preliminary probability estimate. Questions with near-certain outcomes ($<2\%$ or $>98\%$) are flagged to avoid wasting forecasting resources on trivial outcomes. This is a soft signal, not a hard gate.

4.8 Stage 05: Adversarial Peer Review

Verified questions undergo three-phase adversarial review, each using `agent_map()` with web search:

05a: Substantive review. An agent acts as an expert peer reviewer, identifying conceptual, methodological, or practical issues. Each question receives an assessment of PASS, NEEDS_REVISION, or FATAL_FLAW, along with detailed review notes and supporting evidence from web research.

05b: Edge case analysis. A separate agent identifies scenarios where the question breaks or becomes ambiguous—extreme but plausible outcomes, technical loopholes, resolution source failures—and rates overall risk as LOW, MEDIUM, or HIGH.

05c: Revision or kill. Questions flagged by either 05a (NEEDS_REVISION or FATAL_FLAW) or 05b (HIGH risk) are sent to a revision agent that either produces a corrected version addressing all concerns or recommends termination with a detailed rationale. Questions with verdict KILL are excluded from forecasting.

4.9 Stage 06: Ensemble Forecasting

Surviving questions (status PASS or REVISED) are forecasted using FutureSearch’s `forecast()` operation, which implements an internally managed ensemble of multiple LLM forecasters. The operation returns a calibrated probability estimate (0–100) and a detailed rationale for each question. For REVISED questions, the revised text is used as the primary question.

4.10 Stage 06b: Decomposition-and-Research Re-Forecast

The top $n = 2$ questions by priority score receive deeper analysis. (We call this the *decomposition-and-research* re-forecast to distinguish it from the gated *deep-research* stage 06g of Section 4.13; both use web research, but 06b runs the FutureSearch decomposition workflow while 06g commissions an external long-horizon research report.) For each question, the pipeline:

1. **Decomposes** the question into $k = 5$ independent research sub-questions using a `single_agent()` call.
2. **Researches** each sub-question in parallel via `agent_map()` with web search, producing evidence-based answers.
3. **Re-forecasts** by synthesizing all sub-question answers into an updated probability estimate, explaining how each piece of evidence affects the overall estimate.

This implements the “research and decompose” approach described in Bosse et al. [2026], where breaking a complex question into tractable sub-questions and researching each independently can improve calibration.

4.11 Stage 06c: Probabilistic Component Analysis

Building on the qualitative decomposition from Stage 06b, this stage identifies $c = 4$ probabilistic components—sub-events whose probabilities can be mechanically combined to estimate the parent question’s probability. For each question, the system:

1. **Identifies components:** A `single_agent()` determines the key sub-events, their roles in the probability structure, expected dependency patterns, and an explicit combination formula (e.g., $P(\text{YES}) = P(C_1) \cdot P(C_2|C_1)$).
2. **Forecasts components:** Each component is independently forecasted via `agent_map()` with web search.

The combination type is classified as one of: *sequential chain* (events must happen in sequence), *disjunctive paths* (any path suffices), *weighted mixture*, or *hybrid*.

4.12 Stage 06d: Multi-Method Reconciliation

The final forecasting stage reconciles the multiple estimates produced in Stages 06–06c. An `agent_map()` operation processes each question through a five-step reconciliation:

1. **Bottom-up estimate:** Mechanical combination of component probabilities using the formula from Stage 06c.
2. **Dependency adjustment:** Correction for identified conditional dependencies between components.
3. **Structure adjustment:** Check for conceptual issues in the decomposition that might bias the estimate.
4. **Holistic estimate:** An independent probability estimate produced *without* reference to the decomposition.
5. **Reconciliation:** Divergence between bottom-up and holistic estimates is analyzed, and a final probability is produced with explicit rationale.

This multi-method approach—combining ensemble forecasting, research-informed re-forecasting, structured decomposition, and holistic estimation—provides robustness against any single method’s systematic biases.

4.13 Stages 06f–06g: Optional Cross-Model Second Opinions

Two optional stages obtain a forecast from a *different* model family than the FutureSearch ensemble, providing an independent check on the pipeline’s reconciled estimate. Both write their probability, verdict (AGREE/DISAGREE), and critique back into the question record for display in the report.

06f: Local-CLI second opinion. A locally-run agent (the Claude CLI, “Fable”) is re-asked each top-priority question—given its full context and the pipeline’s reconciled forecast—to produce an independent probability and a short critique. Because the call runs against a local CLI rather than the metered FutureSearch API, its marginal cost is effectively zero, making it cheap to run on every deployment.

06g: Deep-research re-forecast (gated). An optional stage commissions a long-horizon deep-research report from an external research agent (by default Google’s **deep-research-max** model) for each top-priority question, then folds that report into the 06f second-opinion call. Because deep-research-max calls are expensive (Section 7.7), this stage is gated behind an explicit `--deep-research` flag and a `DEEP_RESEARCH_MODEL` environment override; it is off by default. Section 7.6 reports a variance-controlled experiment quantifying the effect of this stage on a forecast.

4.14 Stages 06e–08: Report Generation

Two support stages precede the reports. **Stage 06e (seed-passage extraction)** locates, for each forecasted question, the verbatim passage of the source paper that motivated it, together with the corresponding PDF page, so the report can show a question’s provenance back to the exact claim in the seed. **Stage 07q (per-question pages)** then emits a standalone HTML page per question—a table of contents plus every section (decomposition, components, reconciliation, second opinions) expanded—for direct linking and archival.

Per-project reports (Stage 07) and a global cross-project report (Stage 08) are generated as self-contained interactive HTML files. Reports include:

- Sortable/filterable question cards with probability estimates, priority breakdowns, and full provenance.
- Expandable sections showing decomposition trees, component forecasts, and reconciliation analysis.
- Cross-project duplicate detection (string similarity at $\tau = 0.78$ in the global report).
- A methodology summary with links to source papers.

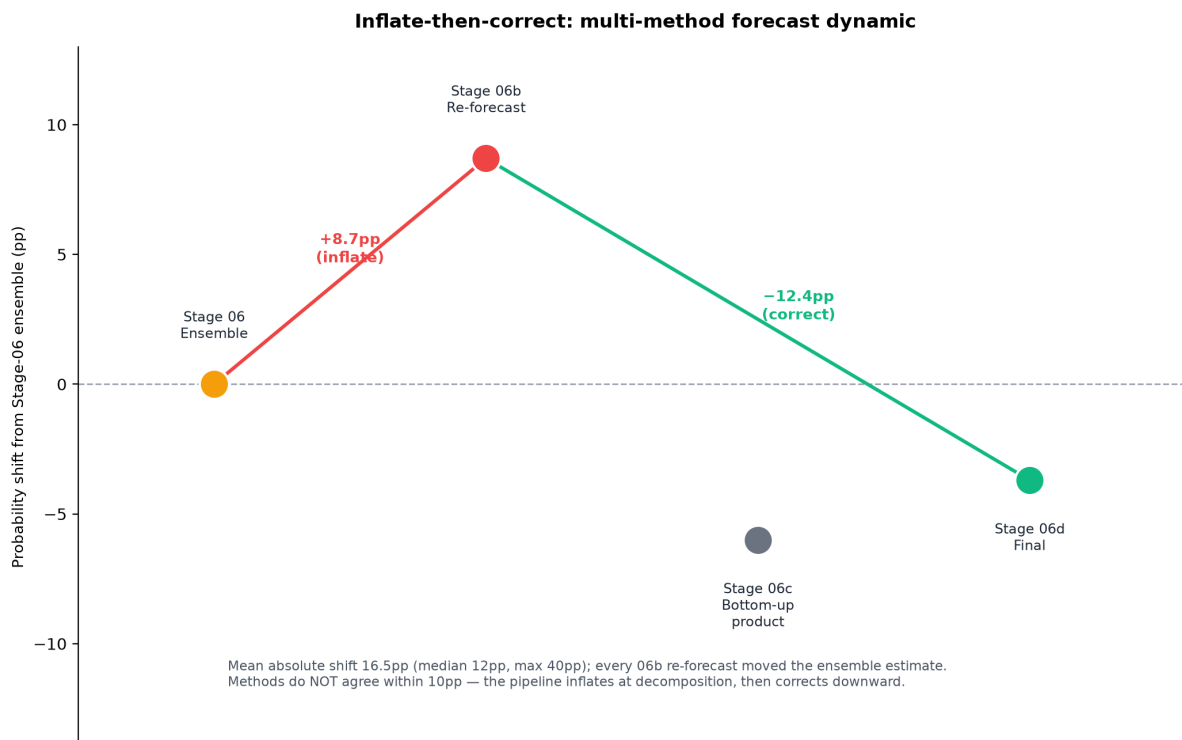


Figure 3: Multi-method forecast dynamic. The four estimation methods do *not* converge. The Stage-06b decomposition-and-research re-forecast inflates the Stage-06 ensemble estimate (signed mean +8.7pp; mean absolute shift 16.5pp, median 12pp, max 40pp), and Stage-06d reconciliation then corrects downward (−12.4pp) toward the mechanical bottom-up component (06c). This inflate-then-correct pattern—not the within-10pp agreement an earlier draft claimed—is the observed behavior (Section 7.3).

- An “Explored Proto-Questions” section showing all candidate questions that were filtered at early stages, with their quality scores, enabling transparency about what the pipeline considered and rejected.

Reports are published to GitHub Pages for sharing and review. Each question card includes a stable anchor link (derived from the question text) that enables direct deep-linking to specific questions from external platforms. A redesign of both report layers on 2026-06-10 made the reconciled probability the headline element of each question card and added client-side full-text search, deep links that auto-expand collapsed sections, mobile layout fixes, and accessibility improvements. The same maintenance pass fixed sixteen independently verified pipeline bugs—including silently mislabeled stage-05 review failures (see Section 8), a stage-02c overwrite of stored proto-question artifacts (see Section 7), and a market close-date extractor that took the first date appearing in the question text; the audited list is in `docs/overnight/HANDOFF.md` in the project repository.

5 Adaptive Effort Allocation

A key design choice is *adaptive effort allocation*, which directs more computational resources (and cost) toward higher-priority questions. The system supports four effort modes:

- **max**: Highest-capability model (Claude Opus) at every stage. Most expensive (~\$20/question).
- **auto**: High effort with model selection delegated to FutureSearch (~\$15/question).
- **adaptive**: Two-dimensional scaling—per-stage baselines combined with per-question scaling by priority score.

- **low:** Minimal-cost model (Gemini Flash) everywhere ($\sim \$1\text{--}2/\text{question}$).

These are nominal per-question model-tier rates. The settled-billing audit of Section 7.7 shows the realized cost of a low-effort paper is dominated not by the base model tier but by the decomposition-and-research re-forecast in Stage 06b ($\approx \$15.62$ of $\approx \$25$ per paper), which the per-stage cost fields under-report while charges settle.

In adaptive mode, the effort function is:

$$\text{effort}(s, q) = \begin{cases} \text{baseline}(s) & \text{if } \text{priority}(q) \geq 75 \\ \min(\text{baseline}(s), \text{MEDIUM}) & \text{if } 55 \leq \text{priority}(q) < 75 \\ \text{LOW} & \text{if } \text{priority}(q) < 55 \end{cases} \quad (2)$$

where s is the pipeline stage and q is the question.

6 Implementation

The pipeline is implemented in Python and orchestrated by a CLI tool (`run_pipeline.py`) that accepts a project name, source document, and configuration parameters. Each stage is an independent script that reads its predecessor’s output and writes structured JSON/CSV files, enabling selective re-execution of individual stages.

FutureSearch SDK. We use five core operations from the FutureSearch Python SDK [Bosse et al., 2026]: `single_agent()` for open-ended generation, `agent_map()` for parallel per-row agent execution with web search, `rank()` for scoring/ranking, `forecast()` for calibrated ensemble probability estimation, and `dedupe()` for semantic deduplication. All operations support Pydantic schema enforcement for structured output.

Prompt management. The sixteen LLM prompts are maintained as tabs of a single Google Doc, which is the ground truth: each is fetched at runtime via the Google Docs API, cached locally, and rendered with its template variables (e.g. `{{resolution_deadline}}`) at invocation time. A local override directory (`prompts/overrides/`) is retained as an escape hatch—an override file, when present, takes precedence and is used verbatim—but no overrides are active in the production configuration. Earlier in development the prompt edits described below were staged as local override files before being folded back into the doc; the doc is now authoritative, with one as-yet-untested stage-03 tightening the sole exception.

Prompt revision: three eras. The generation and filtering prompts passed through three revisions, and separating their effects matters for reading the over-indexing result, which we treat as a headline finding (Section 7.5).

v1 (original). The initial prompts carried a *pro*-benchmark steer: a generation instruction that treated “capability benchmarks” as a first-class question archetype. An audit of the 117 generated questions across five early projects found that 44% of final questions (7/16) and 42% of dismissed proto-questions (42/101) resolved via an AI benchmark, leaderboard, or capability-evaluation score, and that the filtering chain slightly *increased* this share (42%→44%; pure-benchmark 22%→25%) rather than reducing it.

v2 (heavy penalties). A rewrite on 2026-06-10 removed the *pro*-benchmark steer and bolted on heavy anti-benchmark machinery: a hard cap of one benchmark-resolved question in four at generation, real-world archetype floors, score caps at the quality filter (60 for pure-benchmark, 70 for quasi-benchmark) and at the priority filter’s importance dimension (50), a mechanical resolution-source-derived topic-cluster labeling scheme, fragile-source kill rules, atomicity rules against compound multi-AND questions, and a current state-of-the-art check against already-resolved thresholds.

Current (de-steered neutral). A controlled 3-way test on a single seed—the `lyptus_cyber_horizons` paper, itself about cyber-capability benchmarks and thus the maximum-temptation case; 12 one-shot questions per arm, benchmark share measured by a regex proxy—showed the penalties were doing little work beyond the steer removal. The benchmark-flavored share fell 42% $\rightarrow$ 17% when the single pro-benchmark line was deleted (v1 $\rightarrow$ neutral), and only a further 17% $\rightarrow$ 8% when the full v2 penalty scaffolding was added on top—within sampling noise, at the cost of six prompts of added complexity. On that evidence the production prompts were de-steered (commit `ae0c692`): the pro-benchmark steer is gone and the anti-benchmark penalties are stripped from generation, quality, priority, refinement, ambiguity, and importance scoring. The two questions the neutral arm still flagged as “benchmark” were borderline-legitimate governance signals (a government AI-safety institute *publishing* an evaluation), not leaderboard scores. This 3-way test is a single paper, one-shot, regex-scored, $n = 1$ —directional, not a controlled trial.

A separate corpus-wide measurement over the deployed proto-question pools confirms the effect on real deployments; we report it as a headline question-quality result in Section 7.5. The v2 prompts also introduced a new failure mode—over-lawyered, multi-clause resolution criteria (Section 7.4)—addressed by a further, as-yet-untested stage-03 tightening now living in the prompt doc. Per-prompt rationale and verifier notes: `docs/overnight/prompt_v2_notes.md`.

Cost tracking. The pipeline queries the FutureSearch billing API before and after each stage, printing per-stage costs and running balance. A caveat surfaced during the 2026-06-11 deployments: the per-stage cost fields lag, reading \$0.00 while charges settle against the account balance later, so a “\$0.00” stage reading must not be read as free (Section 7.7).

Project structure. Each project is defined by a `project.json` metadata file containing the project name, paper title, description, resolution deadline, source URL, and processing flags. Data flows through a standard directory structure (`projects/<name>/data/`) with numbered output files corresponding to pipeline stages.

7 Results

We deployed the pipeline across eight source papers spanning five research domains; Table 1 summarizes them. Six ran under the v1 prompts and two (`ai_ownership` and `lyptus_cyber_horizons`) under v2, as marked. Unless stated otherwise, the artifact audit below (funnel, review, and multi-method figures) covers the 16-question v1 set, for which the full per-stage artifacts survive; the two v2 deployments underpin the over-indexing result (Section 7.5), the blind comparison (Section 7.4), and the deep-research experiment (Section 7.6).

Data provenance and corrections. All counts here were re-derived from the stored per-stage artifacts (the six v1 deployments audited 2026-06-10; the two v2 deployments added subsequently). Because early drafts predated that audit, several figures were corrected; we collect the corrections here rather than repeating them in the subsections. (i) The stored v1 proto-question pools sum to 148, not the “over 300” an earlier draft reported. (ii) A since-fixed stage-02c bug overwrote the stage-02 proto-question file with a “compatibility copy” of priority-advanced rows in five of the six v1 project directories (all but `cyber`), so parts of the funnel are no longer reproducible from disk; the proto-question counts here come from the unaffected full scored pools written by stage 02b, and some intermediate per-stage counts (including an earlier-reported 15-question quality cut and a 13-question `govai_fellowship` pool) cannot be reproduced. (iii) The adversarial-review pass rate is not the 60–80% an earlier draft claimed: of the 16 audited questions, 6 passed unchanged and 10 were substantively revised, none killed. (iv) The Stage-06 and Stage-06b forecasts do not “agree within 10pp” as an earlier draft stated;

Table 1: Pipeline deployment across eight projects. Era: prompt set used (Section 6). Proto-Qs: stored proto-question pools (full stage-02b scored sets), re-derived from the on-disk artifacts. Final: questions surviving all filters and receiving full forecasts.

Project	Domain	Source	Era	Proto-Qs	Final
cyber	Cybersecurity	Institute for AI Policy and Strategy [2025]	v1	75	3
ai_rd_automation	AI R&D	Kwa et al. [2025]	v1	7	2
biosecurity	Biosecurity	Moulangue [2025]	v1	8	2
80k_biosec_pod	Biosecurity	80,000 Hours Podcast [2025]	v1	19	2
govai_fellowship	AI Governance	Centre for the Governance of AI [2026]	v1	8	7
lab_leaks	Biosecurity	(unpublished)	v1	31	6*
ai_ownership	AI ownership	(unpublished)	v2	16	1
lyptus_cyber_horizons	Cybersecurity	(unpublished)	v2	14	5
Total				178	22*

*The lab_leaks project is unpublished: it is excluded from the global report, from the 16-question v1 audited set, and from the Final total (its six forecasted questions have no stored stage-05 review artifacts). The Final total (22) therefore counts the seven published deployments; the Proto-Qs total (178 = 148 v1 + 30 v2) includes all eight. The two v2 deployments were seeded from internal working papers on AI ownership and on cyber-capability time horizons.

they diverge substantially (Section 7.3). The corrected picture—gates that rarely bind, forecasts that move—is central to the paper’s argument.

Across the six v1 deployments the filtering funnel reduces the stored pool by roughly 90% (148 → 16), with the largest reduction at the stage-02b top- N quality cut. The priority filter (Stage 02c), by contrast, advanced every candidate it received in five of seven audited project runs, binding only in the **cyber** and **lab_leaks** runs. Of the 16 audited questions reaching adversarial review (Stage 05), 6 passed unchanged (37.5%), 10 were substantively revised (62.5%), and none were killed; the review stage thus operates in practice as an *editor* rather than a gate (Section 8.1). Stage-04 verification dropped 1 of 17 questions, and the stage-02d global deduplication removed 0 duplicates in its stored reports.

7.1 Filtering Funnel

For the **cyber** project (the largest), the stored artifacts give the following funnel: 75 proto-questions scored at the quality filter → 5 advanced to priority scoring → 3 above the priority threshold (score ≥ 60) → 3 unchanged through deduplication, verification, and adversarial review (all three substantively revised, none dropped) → 3 fully forecasted questions, all of which received decomposition and re-forecasting (Stage 06b). The probabilistic-component and reconciliation stages (06c–06d) were not run on this project.

7.2 Cross-Project Deduplication

The global report’s cross-project deduplication (at threshold $\tau = 0.78$) identified 2 genuine duplicate pairs across the 16 forecasted questions. These were flagged in the report rather than removed, since questions from different source papers may have subtly different framings that are independently valuable. An earlier threshold of $\tau = 0.50$ produced 82 pairs, the vast majority false positives caused by structural similarity in question formatting.

Within-project near-duplication is the larger problem, and one the local string-similarity check (Stage 02d, `difflib` at $\tau = 0.70$) does not catch: the cyber proto-question pool alone contains 13 reworded variants of a single “will NIST publish an AI-agent-security document” event and 11 variants of “an AI agent places top-3 at a CTF,” yet the stored stage-02d reports removed 0 duplicates. The v2 generation prompt adds a non-duplication rule (one question per

resolution source, no threshold re-rolls), and an embedding-based deduplication is planned.

7.3 Multi-Method Forecast Comparison

The four forecasting methods do not converge; they exhibit an inflate-then-correct dynamic (Figure 3). Measured across the 15 questions with stored decompositions:

- The 06b re-forecast moved *every* estimate relative to the Stage-06 ensemble: mean absolute shift 16.5 percentage points (pp), median 12pp, maximum 40pp, with 8 of 15 questions shifting by ≥ 10 pp. The shift is systematically upward (signed mean +8.7pp; revised up in 11 of 15 cases).
- The bottom-up component estimate (Stage 06c) often diverges substantially from the holistic estimates, typically due to conditional dependencies not fully captured by the component structure.
- For the 10 questions completing reconciliation (Stage 06d; not run on the cyber and lab_leaks projects), the final estimate corrected systematically *downward* relative to the 06b re-forecast (signed mean -12.4 pp), landing between the re-forecast and the mechanical bottom-up component product in 8 of 10 cases.

The pipeline thus exhibits an *inflate-then-correct* dynamic: decomposition research raises probabilities, and the mechanical component combination anchors them back down. We return to the architectural implications in Section 8.1.

7.4 Blind Pipeline-vs-One-Shot Comparison

To probe whether the multi-stage architecture produces better questions than a single model pass, we ran a blinded comparison on the “Diversifying AI Ownership” paper. The high-effort pipeline produced one final question; the same model family (Table 5) produced five questions in a single pass with no web access, locked by commit before any pipeline output existed. All six questions were normalized to a uniform template, shuffled, and scored blind on a 0–100 scale by three independent judge agents (Metaculus-moderator, superforecaster, and policy-researcher personas) with web access for fact-checking. Judges did not know the sources or the mix.

The pipeline question scored 80.0 overall versus a one-shot mean of 60.6; it beat even the best one-shot question (74.3), was ranked first by all three judges, and was the only question with unanimous tournament acceptance. The mechanism was web research: the two worst one-shot questions were factual-currency failures of exactly the class the pipeline’s research stages catch—one was already determined (central banks the question asks about already buy equities in their own-funds portfolios; uncertainty score 14, rejected by all judges) and one was mooted by a December 2025 act codifying the exception it asked about. The pipeline question, built on researched post-cutoff facts, scored 89 on genuine uncertainty. Its weakness was the one the architecture critique predicted: atomicity (57.3, the lowest dimension), reflecting over-lawyered, multi-clause resolution criteria that survive the v2 prompts. Notably, the judges guessed the source correctly 18 of 18 times—pipeline output is stylistically distinguishable by its researched precedents and legal fine print.

Two caveats bound this result. First, $n = 1$ pipeline question: the narrow funnel produced exactly one final question this run, so the comparison cannot support strong claims. Second, the tooling is asymmetric by design (the one-shot had no web access), so the fair reading is “research-backed multi-stage beats raw parametric knowledge,” not “the full architecture is necessary”—a single research-enabled pass was not tested and is the natural next experiment (and the arm against which to validate the consolidated design of Section 8.1). All three judges are also Claude-family agents, so their tastes may correlate with the generator. Artifacts: docs/experiments/oneshot_vs_pipeline_results.md.

7.5 Benchmark Over-Indexing

A recurring failure of automated question generation is *benchmark over-indexing*: producing “will model X reach $Y\%$ on benchmark Z ?” questions in place of decision-relevant real-world outcomes. Such questions are easy to generate and easy to resolve, but they largely re-report a number a public leaderboard already tracks, so they carry little marginal decision value. The over-indexing rate is thus a direct, automatable measure of question *quality*, and it is the pipeline’s clearest quality signal.

We applied a benchmark-pattern classifier (a regex over benchmark, leaderboard, eval-score, and time-horizon phrasings) to the full stage-02b proto-question pools. Under the original v1 prompts, 33 of 148 proto-questions (22%) were benchmark-flavored, concentrated in the most capability-focused papers (`ai_rd_automation` 43%, `govai_fellowship` 38%, `cyber` 27%). Under the v2 prompts, 0 of 30 (0%) were—including on `lyptus_cyber_horizons`, a paper *itself about* cyber-capability time-horizon benchmarks and thus the maximum-temptation case, which instead produced incident, legislation, treaty, and agency-publication questions. Because `lyptus` never ran under v1, the fair v1 comparison is its thematic neighbor `cyber`, at 27% on the same subject matter.

The controlled 3-way test of Section 6 shows that most of this drop comes from *removing* a single pro-benchmark steer line, not from the v2 penalties: on the same seed, one-shot benchmark share fell 42% $\rightarrow$ 17% from the steer removal and only a further 17% $\rightarrow$ 8% from the full penalty scaffolding. The production prompts accordingly keep the steer removed and drop the penalties. Both measurements are directional—the corpus is $n = 30$ over two papers and the 3-way test is a single paper, one-shot, regex-scored—and the cleanest same-paper test was priced out by the per-paper cost (Section 7.7); but they agree in sign and rough magnitude, and the deployed pools confirm the effect where it matters. This is the pipeline’s strongest question-quality result after the blind comparison of Section 7.4.

7.6 Deep-Research Second Opinion (Variance-Controlled)

We emphasize that forecast augmentation is *not* the pipeline’s goal; this experiment is secondary evidence that the plumbing works—that an external research report can be wired into a second opinion and measurably moves it—not a claim to competitive forecast accuracy, which specialists lead. With that framing, we ran a variance-controlled experiment to isolate the effect of the gated deep-research stage (06g, Section 4.13) from the local CLI’s own sampling spread. On a single top-priority question (whether the U.S. federal government will take a $\geq 1\%$ equity stake in a frontier AI lab), the 06f second-opinion call was run three times *without* a deep-research report and three times *with* one, using a fresh CLI session each time:

- **Without deep research:** forecasts of 25, 32, 30 (mean 29.0, range 25–32).
- **With deep research:** forecasts of 42, 37, 37 (mean 38.7, range 37–42).

The two ranges do not overlap (the deep-research minimum, 37, exceeds the no-deep-research maximum, 32), so the report raises the forecast by roughly +10pp (29 $\rightarrow$ 39), an effect that clearly exceeds the ≈ 7 pp CLI sampling band. The shift is a real, repeatable effect rather than noise. Directionally, the deep-research report argued that federal equity stakes are already established (citing a 9.9% Intel stake plus mining and quantum precedents), so extending the pattern to AI is “mechanically likely”—pulling the estimate upward. This is consistent with the blind-experiment finding that research-backed questions beat memory-only ones.

Three caveats apply. First, $n = 1$ question: only this question received the full six-run variance control (the CLI rate-limits under load, and a batch script failed mid-run on the other two), so the +10pp magnitude and even its direction will vary by question. Second, the three deep-research-max calls consumed 5.6M tokens (0 cached) at roughly \$12–21 total—a material fraction of a run’s budget. Third, the deep-research report can itself be wrong; here it had no independent ground truth. Artifacts: `docs/overnight2/deep_research_results.md`.

7.7 Cost Reality

This subsection is the paper’s single accounting of cost; other sections refer back to it. When the 2026-06-11 charges settled against the account balance (the per-stage fields having lagged at \$0.00; Section 6), one low-effort full-pipeline paper cost approximately \$25, of which stage 06b—the decomposition-and-research re-forecast—alone accounted for \$15.62, or 63% of the paper. The remaining cost was distributed across generation (Stage 02, \$3.74), the verification and review chain (Stages 04–05, ≈\$3), the filters (≈\$1.6), and the base ensemble forecast (Stage 06, \$0.16). The gated deep-research stage (06g) is billed separately by Google, at roughly \$4–7 per question. This concentration of cost in stage 06b—which the artifact audit also identifies as one of the lowest-value stages (Section 8.1)—strengthens the case for gating or removing the 06b–06d chain: at ≈\$25 per paper, batch-processing many papers is not economical, whereas cutting that chain would remove the dominant cost driver. Budget reality: `docs/overnight2/HANDOFF.md`.

7.8 Deployment: Prediction-Market Integration

An optional final stage publishes forecasted questions to *Manifold Markets*², a prediction-market platform, closing the loop between the forecast repository and a live crowd. An automated upload script reads the top-ranked questions from the global report (sorted by ITNSSS priority score), uses an LLM to shorten each into a concise market title (≤ 120 characters), checks the Manifold search API for existing duplicates, and creates binary markets via `POST /v0/market`—with the initial probability set to the pipeline’s reconciled forecast, the close date extracted from the question’s resolution timeline, and a rich description carrying the full question text, background, resolution criteria, decomposition sub-questions, component breakdowns, and reconciliation estimates. The resulting market URLs are written back into the question data and displayed as “Bet” badges on the report. This provides a complementary human-forecaster signal against which the pipeline’s automated estimates can eventually be calibrated. As of writing, six markets have been created from top-priority questions across the biosecurity, AI-governance, and AI-automation deployments.

7.9 Case Study: Recursive-Self-Improvement Round (July 2026)

As the first deployment of the de-steered production prompts (Section 6) together with a batch of fourteen targeted prompt fixes (domain de-hardcoding, atomicity and source-metric checks, and reconciliation-prompt corrections), we ran a fresh generation round on the theme of automated AI research and development. The seed is *Import AI* issue 455 on automated AI R&D, supplemented by six sources on lab-automation adoption, compute and productivity statistics, and takeoff-timeline forecasts. The round is configured to generate 45 proto-questions and advance the top twelve to refinement.

Because it runs entirely under the current prompts, this round will provide the first same-configuration corpus for the over-indexing and question-quality measures of Section 7.5 and Section 7.4; results are pending at submission.

8 Discussion

Automation vs. oversight. The pipeline is designed to run end-to-end without manual intervention, but produces HTML reports with full provenance at each stage to enable human review. The “Explored Proto-Questions” section—showing all candidates that were generated but filtered—provides transparency about the pipeline’s coverage and decisions. We envision a workflow where the pipeline runs automatically but a human analyst reviews the final report before publication.

²<https://manifold.markets>

Calibration. We do not yet have resolution data to evaluate calibration empirically, as the questions have resolution deadlines in late 2027. Future work will compare pipeline forecasts against actual outcomes and against human forecaster baselines.

Cost–quality trade-offs. The adaptive effort mode provides a reasonable middle ground: high-priority questions receive full computational resources while lower-priority questions are processed more cheaply. However, the settled-billing audit of Section 7.7 revised the per-paper cost sharply upward from earlier estimates, and concentrated it in the single decomposition stage. At that price, batch-processing many papers is not economical without either gating the expensive 06b–06d chain or adopting the cheaper consolidated architecture of Section 8.1.

Validity note: silent review failures. Until a bug fix on 2026-06-10, stage-05 agent failures (agents that crashed or returned no output) were silently labeled PASS. Questions processed before the fix and described as adversarially reviewed may therefore never have been reviewed; failed reviews are now labeled NOT_EVALUATED or REVISION_FAILED and surfaced in the reports (whether such questions should be blocked from forecasting remains an open policy decision). For the 16 questions in the audited set, the stored stage-05 artifacts all contain substantive review text; but the six lab_leaks questions were forecast without stored review artifacts, and the blanket claim that every forecasted question passed adversarial review cannot be made for deployments predating the fix.

Limitations. The pipeline inherits the limitations of its underlying LLM agents: potential for hallucination in research summaries, sensitivity to prompt wording, and possible systematic biases in probability estimation. Sensitivity to prompt wording is the sharpest of these, and the benchmark over-indexing of Section 7.5 is the clearest case: it was *not* an intrinsic model bias but an artifact induced by a single pro-benchmark steer line in the v1 prompt, and it largely vanished once that line was removed. It is thus better read as a caution about prompt authoring than about model priors—and the de-steer traded one issue for another, since the heavily penalized v2 prompt introduced over-lawyered resolution criteria (visible as the low atomicity score in Section 7.4). Prompt-level fixes move the failure around; they do not eliminate it. The adversarial review stage mitigates some of these issues but cannot eliminate them entirely. Additionally, the quality of proto-questions depends heavily on the richness of falsifiable claims in the source paper; papers with few concrete predictions yield fewer viable forecasting questions. Finally, the “no manual intervention” claim of Section 1 held only partially before the 2026-06-10 bug-fix sweep: fresh-project runs failed at stage-input validation until that fix.

8.1 Architectural Simplification

A post-deployment audit of the stored artifacts across all seven project runs (`docs/pipeline_architecture_critique.md` in the project repository) found that most of the pipeline’s quality gates never bind. Stage-04 triple verification dropped 1 of 17 questions—with the ambiguity and resolvability gates tripping redundantly on the same item, and the triviality screen firing on none. The stage-02d global deduplication removed 0 duplicates in its stored reports, while 13 NIST-variant and 11 CTF-variant near-duplicates passed through. The stage-02c priority filter advanced every candidate in five of seven runs, and its diversity constraint never fired. The stage-05 kill path fired 0 of 16 times—yet stage 05 substantively rewrote 10 of 16 questions, including factual corrections: it is the pipeline’s most valuable stage, but as an *editor*, not a gate. The same four properties (resolution ambiguity, triviality, source fragility, benchmark dependence) are re-judged four to six times across prompts. And the 06b–06d chain forms the inflate-then-correct pattern of Section 7: 06b shifts forecasts +8.7pp on average, and 06c–06d correct −12.4pp back toward a mechanical bottom-up product—suggesting the disciplined fix is

base-rate scaffolding in the 06b re-forecast prompt rather than two standing correction stages. The case for trimming this chain is sharpened by the settled-billing audit (Section 7.7): the 06b decomposition stage is by far the most expensive part of a run, so the architecture’s lowest-value stages are also its costliest.

The root cause is an architectural asymmetry: this is a narrow-funnel adaptation of a wide-funnel design. Bosse et al. [2026] run on the order of 2,500 seed papers $\rightarrow$ 14,000 proto-questions $\rightarrow$ 1,500 final questions, and can afford cheap, discard-only gates. This pipeline runs one paper $\rightarrow$ $\sim$ 50 proto-questions $\rightarrow$ 2–7 final questions, and cannot afford to discard—so it accreted editing and correction stages (defensible) while also re-judging the same properties repeatedly (not defensible).

As future work, we plan to evaluate a consolidated five-pass architecture: (1) generation unchanged; (2) a single combined quality-plus-priority ranking with selection in code (free string-similarity deduplication, a quality threshold, a continuous priority composite, and the cluster cap); (3) combined refinement and verification per question, emitting a deliberately loose `needs_review` flag; (4) adversarial review only on flagged questions; and (5) forecasting with a single decompose–research–re-forecast pass whose probabilistic component chain is combined mechanically in code, invoking the reconciliation agent only when the holistic and bottom-up estimates diverge by more than 15pp. The audit predicts roughly $0.45\times$ the current cost while preserving every empirically active component. Before switching, we will run a blind validation experiment on one fresh seed paper: both architectures with identical configuration, provenance-stripped and shuffled outputs rated blind by human and cross-vendor LLM judges, an objective defect audit applying the existing stage-04 prompts post hoc to both final sets, and an independent ensemble forecast per question as a sanity proxy. The blind-comparison methodology has already been exercised once, in the pipeline-vs-one-shot study of Section 7.4; the consolidated-vs-current A/B is the natural next application, with a single research-enabled pass (rather than a no-web one-shot) as the comparison arm.

9 Conclusion

We have presented a 16-stage automated pipeline (plus two optional cross-model second-opinion stages) that transforms research papers into decision-relevant binary forecasting questions. Its product is the *questions*, and the two strongest results are about question quality: a blinded comparison found the pipeline’s question outscored a one-shot model’s (80.0 vs. 60.6, ranked first by all three judges), and a corpus audit found that benchmark over-indexing—the main quality failure of naive generation—fell from 22% to 0% once a single pro-benchmark prompt line was removed. The system combines LLM-based generation with structured quality gates, multi-dimensional priority scoring, adversarial review, and multi-method probabilistic forecasting, but the post-deployment artifact audit shows that most hard gates rarely bind in the narrow per-paper funnel: quality is enforced chiefly through refinement and adversarial revision, and the forecasting chain mostly inflates then corrects. The interactive reports and full provenance tracking enable human oversight without requiring manual question generation. Future work will focus on empirical calibration evaluation once questions resolve, expanding the pipeline to additional source types (podcasts, policy documents, technical reports), and validating the consolidated five-pass architecture of Section 8.1 against the current design in a blind one-paper experiment.

References

80,000 Hours Podcast. Richard moulange on ai, bioweapons, and biorisk, 2025. URL <https://80000hours.org/podcast/episodes/richard-moulange-ai-bioweapons-biorisk/>.

- Kenneth J Arrow, Robert Forsythe, Michael Gorham, et al. The promise of prediction markets. *Science*, 320(5878):877–878, 2008.
- Nikos I. Bosse et al. Automating forecasting question generation and resolution for ai evaluation. *arXiv preprint arXiv:2601.22444*, 2026. URL <https://arxiv.org/abs/2601.22444>. FutureSearch; released 2026-01-30.
- Centre for the Governance of AI. Winter fellowship 2026, 2026. URL <https://www.governance.ai/post/winter-fellowship-2026>.
- Danny Halawi, Fred Schneider, Tim Ye, et al. Approaching human-level forecasting with language models. *arXiv preprint arXiv:2402.18563*, 2024.
- Institute for AI Policy and Strategy. Highly autonomous cyber-capable agents. Technical report, 2025. URL <https://www.iaps.ai/research/highly-autonomous-cyber-capable-agents>.
- Ghader Kurdi, Jared Leo, Bijan Parsia, Uli Sattler, and Salam Al-Emari. A systematic review of automatic question generation for educational purposes. *International Journal of Artificial Intelligence in Education*, 30:121–204, 2020.
- Thomas Kwa et al. Measuring ai’s ability to complete long tasks. *arXiv preprint arXiv:2603.03992*, 2025. URL <https://arxiv.org/abs/2603.03992>.
- Richard Moulange. Reasons to be pessimistic and optimistic about ai bioweapons risk, 2025. URL <https://www.owlposting.com/p/reasons-to-be-pessimistic-and-optimistic>.
- Open Philanthropy. Importance, tractability, and neglectedness, 2016. URL <https://www.openphilanthropy.org/research/some-useful-heuristics/>. Accessed 2026-04-14.
- Toby Ord. *The Precipice: Existential Risk and the Future of Humanity*. Hachette Books, 2020.
- Philipp Schoenegger, Peter S. Park, Ezra Karger, and Philip E. Tetlock. Wisdom of the silicon crowd: LLM ensemble prediction capabilities match human crowd accuracy. *Science Advances*, 2024.
- Philip E Tetlock and Dan Gardner. *Superforecasting: The Art and Science of Prediction*. Crown, 2015.
- Lidan Wang, Jimmy Lin, and Donald Metzler. A cascade ranking model for efficient ranked retrieval. In *Proceedings of SIGIR*, pages 105–114, 2011.

A ITNSSS Priority Score Components

Table 2 details the six components of the priority scoring framework used in Stage 02c.

B Pipeline Stage Summary

Table 3 provides a concise summary of all pipeline stages with their FutureSearch operations and quality gates.

Table 2: ITNSSS priority score components. Each is scored 0–100 by an LLM agent with web search access.

Component	Source	Weight	Description
Importance	ITN	0.25	Decision-relevance: would knowing the answer change resource allocation or policy?
Neglectedness	ITN	0.25	Marginal information value: how much existing research, data, or forecasting covers this question?
Tractability	ITN	0.20	Feasibility: are there base rates, resolution sources, and analogues that make meaningful forecasting possible?
Soon	Ord	0.10	Temporal urgency: will this question resolve in the near term?
Sudden	Ord	0.10	Discreteness: is the outcome a sudden event rather than a gradual trend?
Sharp	Ord	0.10	Surprise potential: might the outcome occur without clear warning signals?

Table 3: Pipeline stage summary. Operations refer to FutureSearch SDK functions.

Stage	Name	Operation	Gate	Output
01	Text extraction	Local	–	Plain text
02	Proto-generation	single_agent	Top m /chunk	Proto-questions
02b	Quality filter	rank	Top N by score	Quality-scored Qs
02c	Priority filter	agent_map	Score ≥ 60 , diversity	Priority-scored Qs
02d	Global dedup	diffib/dedupe	Similarity $< \tau$	Deduplicated Qs
03	Refinement	agent_map	–	Refined questions
04	Verification	rank + agent_map	Ambiguity ≥ 60 , resolvable	Verified questions
05	Adversarial review	agent_map $\times 3$	Pass/revise/kill	Reviewed questions
06	Forecasting	forecast	–	Probability estimates
06b	Decomposition	single_agent + agent_map	–	Sub-question research
06c	Component analysis	single_agent + agent_map	–	Component forecasts
06d	Reconciliation	agent_map	–	Final estimates
06e	Seed-passage extract	Local	–	Provenance passages
06f	Second opinion	Local CLI	–	Cross-model critique [†]
06g	Deep research	External agent	–	Research-backed re-forecast [†]
07q	Per-question pages	Local	–	Standalone HTML pages
07–08	Reporting	Local	–	Interactive HTML

[†]Stages 06f and 06g are optional cross-model second-opinion stages (Section 4.13); 06g is off by default and gated behind a `--deep-research` flag.